

ML Projects + LeetGPU Mastery Plan

Akshith Macharla — 10 Week Roadmap

****Daily Structure:****

- Morning (2-3 hrs): LeetGPU problem — solve in BOTH PyTorch and CUDA
- Afternoon/Evening (3-5 hrs): Project work
- Night (30 min): Document what you learned in repo README

WEEK 1: PyTorch Boot Camp (No project yet — build foundations)

Day 1 — Vector Addition + Matrix Addition

- ****LeetGPU (CUDA):**** Vector Addition, Matrix Addition
- ****PyTorch:**** Implement both using torch tensors, understand broadcasting
- ****Project Work:**** Set up all 4 project repos on GitHub. Install PyTorch, W&B, create conda environments

Day 2 — ReLU + Leaky ReLU + Sigmoid

- ****LeetGPU (CUDA):**** ReLU, Leaky ReLU, Sigmoid Activation
- ****PyTorch:**** Implement all three as custom `torch.autograd.Function` with forward AND backward
- ****Project Work:**** Train a 2-layer MLP on MNIST from scratch. Raw training loop — no Trainer, no Lightning. Just `for epoch`, `for batch`, `loss.backward()`, `optimizer.step()`

Day 3 — Matrix Multiplication + Matrix Transpose

- ****LeetGPU (CUDA):**** Matrix Multiplication, Matrix Transpose

- **PyTorch:** Implement `nn.Linear` from scratch using `torch.matmul`
- **Project Work:** Replace your MLP's `nn.Linear` with your custom linear layer. Verify same results

Day 4 — Matrix Copy + Reverse Array

- **LeetGPU (CUDA):** Matrix Copy, Reverse Array
- **PyTorch:** Practice tensor slicing, cloning, `.contiguous()`, memory layout
- **Project Work:** Add learning rate scheduler + W&B logging to your MNIST training. Log loss, accuracy, learning rate per epoch

Day 5 — SiLU + SwiGLU + GELU

- **LeetGPU (CUDA):** Sigmoid Linear Unit, Swish-Gated Linear Unit, GELU
- **PyTorch:** Implement all three as custom modules
- **Project Work:** Train ResNet-18 on CIFAR-10. Use `torchvision.models.resnet18(pretrained=False)`. Add data augmentation (RandomCrop, RandomHorizontalFlip). Log to W&B

Day 6 — Reduction + Dot Product

- **LeetGPU (CUDA):** Reduction, Dot Product
- **PyTorch:** Understand `torch.sum`, `torch.mean` reduction dims, implement dot product manually
- **Project Work:** Fine-tune pretrained BERT on a simple binary text classification task (SST-2 from HuggingFace datasets). Use HuggingFace tokenizer but write YOUR OWN training loop — not Trainer API

Day 7 — Categorical Cross Entropy + MSE

- **LeetGPU (CUDA):** Categorical Cross Entropy Loss, Mean Squared Error

- **PyTorch:** Implement both loss functions from scratch. Compare with `nn.CrossEntropyLoss` and `nn.MSELoss`
- **Project Work:** Finish BERT fine-tuning. Evaluate with accuracy, F1. Save checkpoint. You now have end-to-end training experience.

Week 1 Checkpoint: You can write a PyTorch training loop from scratch, you understand forward/backward/optimizer, you've trained CNN + Transformer. 11 LeetCode problems done.

WEEKS 2-3: Project 1 — NLP Transformer Fine-tuning + Deploy

Day 8 — Softmax

- **LeetCode (CUDA):** Softmax
- **PyTorch:** Implement numerically stable softmax from scratch (subtract max trick)
- **Project Work:** Set up Project 1 repo. Choose dataset (Toxic Comment multi-label or arXiv paper classification). Download, explore, write `Dataset` class

Day 9 — Batch Normalization + RMS Normalization

- **LeetCode (CUDA):** Batch Normalization, RMS Normalization
- **PyTorch:** Implement both from scratch, understand running stats vs batch stats
- **Project Work:** Load pretrained DeBERTa-v3-base. Tokenize dataset. Build DataLoader with padding and attention masks

Day 10 — Softmax Attention

- **LeetCode (CUDA):** Softmax Attention
- **PyTorch:** Implement single-head attention from scratch: $Q @ K^T / \sqrt{d} \rightarrow \text{softmax} \rightarrow @ V$

- ****Project Work:**** Write training loop for DeBERTa fine-tuning. BCE loss (multi-label). AdamW optimizer with linear warmup scheduler. Log to W&B

Day 11 — Color Inversion + RGB to Grayscale

- ****LeetGPU (CUDA):**** Color Inversion, RGB to Grayscale
- ****PyTorch:**** Implement as tensor operations
- ****Project Work:**** Train baseline model. Record baseline metrics: per-class F1, macro F1, micro F1, precision, recall. This is your benchmark to beat

Day 12 — Value Clipping + Interleave Arrays

- ****LeetGPU (CUDA):**** Value Clipping, Interleave Arrays
- ****PyTorch:**** Implement `torch.clamp` equivalent, understand gradient clipping
- ****Project Work:**** Ablation 1 — Learning rate sweep (1e-5, 2e-5, 3e-5, 5e-5). Log all runs to W&B. Pick best

Day 13 — Rotary Positional Embedding

- ****LeetGPU (CUDA):**** Rotary Positional Embedding
- ****PyTorch:**** Implement RoPE from scratch — understand sin/cos position encoding
- ****Project Work:**** Ablation 2 — Frozen vs unfrozen base layers. Compare training speed and final metrics. Ablation 3 — Label smoothing (0.0 vs 0.1 vs 0.2)

Day 14 — 1D Convolution

- ****LeetGPU (CUDA):**** 1D Convolution
- ****PyTorch:**** Implement 1D conv from scratch using torch.unfold
- ****Project Work:**** Write evaluation script. Generate confusion matrix, per-class metrics table. Create W&B summary dashboard with all ablation results compared

Day 15 — Prefix Sum

- **LeetGPU (CUDA):** Prefix Sum
- **PyTorch:** Implement `torch.cumsum` equivalent
- **Project Work:** Build FastAPI inference endpoint. Load saved model checkpoint, accept text input, return predictions with confidence scores

Day 16 — Count Array Element + Subarray Sum

- **LeetGPU (CUDA):** Count Array Element, Subarray Sum
- **PyTorch:** Practice reduction operations
- **Project Work:** Dockerize the FastAPI app. Write Dockerfile, test locally. Push to Docker Hub or AWS ECR

Day 17 — Top K Selection

- **LeetGPU (CUDA):** Top K Selection
- **PyTorch:** Understand `torch.topk`, implement beam search concepts
- **Project Work:** Write comprehensive README: problem description, approach, ablation results table, metrics, how to run, how to deploy. Clean code. Push to GitHub

Week 2-3 Checkpoint: Project 1 complete. You have a fine-tuned transformer with ablation studies, evaluation metrics, and deployed endpoint. 23 LeetGPU problems done.

WEEKS 4-5: Project 2 — Vision Training (ResNet + ViT) + GPU Profiling

Day 18 — 2D Convolution

- **LeetGPU (CUDA):** 2D Convolution

- **PyTorch:** Implement 2D conv from scratch using `torch.nn.functional.unfold`
- **Project Work:** Set up Project 2 repo. Choose dataset (Stanford Cars, Food-101, or Oxford Pets). Write custom Dataset with augmentation pipeline

Day 19 — 2D Max Pooling

- **LeetGPU (CUDA):** 2D Max Pooling
- **PyTorch:** Implement max pooling from scratch
- **Project Work:** Build ResNet-18 from scratch in PyTorch — don't use torchvision. Write every `nn.Conv2d`, `nn.BatchNorm2d`, skip connections yourself. This is critical

Day 20 — Gaussian Blur

- **LeetGPU (CUDA):** Gaussian Blur
- **PyTorch:** Implement as 2D convolution with Gaussian kernel
- **Project Work:** Write training loop for ResNet from scratch. Mixed precision with `torch.cuda.amp`. Cosine annealing LR scheduler. W&B logging

Day 21 — Multi-Head Attention

- **LeetGPU (CUDA):** Multi-Head Attention
- **PyTorch:** Implement multi-head attention from scratch — split heads, attention, concat, project
- **Project Work:** Train ResNet from scratch on your dataset. Record training curves, final accuracy, training time

Day 22 — 3D Convolution

- **LeetGPU (CUDA):** 3D Convolution
- **PyTorch:** Implement `nn.Conv3d` equivalent
- **Project Work:** Load pretrained ViT (`vit_base_patch16_224` from `timm`). Fine-tune on same dataset. Same training loop structure, same hyperparameters where applicable

Day 23 — Histogramming

- **LeetGPU (CUDA):** Histogramming
- **PyTorch:** Practice `torch.histc`, understand atomic operations concept
- **Project Work:** GPU profiling comparison. Profile BOTH ResNet and ViT training with `torch.profiler` (or Nsight if you have access). Compare: GPU utilization, memory usage, throughput (images/sec), time per layer

Day 24 — GEMM (FP16)

- **LeetGPU (CUDA):** General Matrix Multiplication (GEMM)
- **PyTorch:** Understand `torch.matmul` with FP16 tensors, Tensor Core usage
- **Project Work:** Build comparison analysis. Side-by-side: ResNet vs ViT — accuracy, training time, compute cost, GPU memory, profiling results. Create visualization in matplotlib

Day 25 — Sparse Matrix-Vector Multiplication

- **LeetGPU (CUDA):** Sparse Matrix-Vector Multiplication
- **PyTorch:** Practice `torch.sparse`, understand CSR format
- **Project Work:** Write comprehensive README with profiling charts, comparison table, training curves. Your GPU profiling angle is what makes this project unique — highlight it

Day 26 — Batched Matrix Multiplication

- **LeetGPU (CUDA):** Batched Matrix Multiplication
- **PyTorch:** Implement batched matmul, understand `torch.bmm` vs `torch.matmul` broadcasting
- **Project Work:** Final cleanup. Ensure reproducibility (set seeds, document exact commands). Push to GitHub

****Week 4-5 Checkpoint:**** Project 2 complete. You've trained CNN + ViT from scratch with GPU profiling comparison. 32 LeetCode problems done.

WEEKS 6-7: Project 3 — LLM Fine-tuning with LoRA + Deploy

Day 27 — LoRA Linear

- ****LeetCode (CUDA):**** LoRA Linear
- ****PyTorch:**** Implement LoRA layer from scratch: $\text{output} = W @ x + (B @ A) @ x * \text{scaling}$
- ****Project Work:**** Set up Project 3 repo. Choose base model (Mistral-7B or Llama-3-8B). Choose task (instruction following on Alpaca, or domain Q&A)

Day 28 — Causal Self-Attention

- ****LeetCode (CUDA):**** Causal Self-Attention
- ****PyTorch:**** Implement causal attention with mask from scratch
- ****Project Work:**** Set up QLoRA with bitsandbytes. Load model in 4-bit. Apply LoRA adapters to attention layers. Understand which layers to target

Day 29 — Grouped Query Attention

- ****LeetCode (CUDA):**** Grouped Query Attention
- ****PyTorch:**** Implement GQA — understand KV head sharing
- ****Project Work:**** Prepare training data. Format as instruction-response pairs. Tokenize with proper chat template. Build DataLoader

Day 30 — INT8 Quantized MatMul

- ****LeetCode (CUDA):**** INT8 Quantized MatMul

- **PyTorch:** Understand `torch.quantization`, dynamic vs static quantization
- **Project Work:** Write training loop for LoRA fine-tuning. Gradient accumulation (to simulate larger batch). Log to W&B

Day 31 — INT4 Weight-Only Quantized MatMul

- **LeetGPU (CUDA):** INT4 Weight-Only Quantized MatMul
- **PyTorch:** Understand 4-bit quantization schemes (NF4, FP4)
- **Project Work:** Train with LoRA rank=4. Record baseline metrics. Then ablation: rank 4 vs 16 vs 64. Compare: final loss, task metrics, training time, GPU memory

Day 32 — Weight Dequantization

- **LeetGPU (CUDA):** Weight Dequantization
- **PyTorch:** Implement dequantize from INT4 to FP16 manually
- **Project Work:** Build evaluation harness. Task-specific metrics: ROUGE (for generation), exact match (for Q&A), or custom rubric. NOT just perplexity — actual task performance

Day 33 — MoE Top-K Gating

- **LeetGPU (CUDA):** MoE Top-K Gating
- **PyTorch:** Implement top-k expert selection with load balancing loss
- **Project Work:** Compare: base model vs LoRA-4 vs LoRA-16 vs LoRA-64 on evaluation harness. Create comparison table

Day 34 — Top-p Sampling

- **LeetGPU (CUDA):** Top-p Sampling
- **PyTorch:** Implement nucleus sampling from scratch for text generation
- **Project Work:** Deploy with vLLM. Set up vLLM server with LoRA adapter loaded. Benchmark: throughput (tokens/sec), TTFT, latency at different batch sizes

Day 35 — GPT-2 Transformer Block

- **LeetGPU (CUDA):** GPT-2 Transformer Block
- **PyTorch:** Implement a complete transformer decoder block from scratch: LayerNorm → MHA → residual → LayerNorm → FFN → residual
- **Project Work:** Compare serving: vLLM vs your own FastAPI + HuggingFace generate. Measure latency and throughput differences. Write README with full results

Day 36 — Linear Self-Attention

- **LeetGPU (CUDA):** Linear Self-Attention
- **PyTorch:** Implement linear attention approximation
- **Project Work:** Final cleanup, documentation, push to GitHub

Week 6-7 Checkpoint: Project 3 complete. You've fine-tuned an LLM with LoRA, built evaluation harness, benchmarked quantization, deployed with vLLM. 43 LeetGPU problems done.

WEEKS 8-10: Project 4 — Distributed Training Framework

Day 37 — FP16 Batched Matrix Multiplication

- **LeetGPU (CUDA):** FP16 Batched Matrix Multiplication
- **PyTorch:** Understand AMP, `torch.cuda.amp.autocast`, `GradScaler`
- **Project Work:** Set up Project 4 repo. Train a simple model (ResNet-18 on CIFAR-10) on single GPU as baseline. Record throughput, time per epoch

Day 38 — FP16 Dot Product

- **LeetGPU (CUDA):** FP16 Dot Product
- **PyTorch:** Compare FP32 vs FP16 dot product precision and speed
- **Project Work:** Implement `DistributedDataParallel` (DDP). Wrap model, use `DistributedSampler`. Train on 2 GPUs. Measure speedup vs single GPU

Day 39 — Sliding Window Self-Attention

- **LeetGPU (CUDA):** Sliding Window Self-Attention
- **PyTorch:** Implement sliding window attention with masking
- **Project Work:** Add mixed precision training (AMP) to DDP setup. Benchmark: DDP + AMP vs DDP + FP32 vs single GPU

Day 40 — Attention with Linear Biases (ALiBi)

- **LeetGPU (CUDA):** Attention with Linear Biases
- **PyTorch:** Implement ALiBi position encoding
- **Project Work:** Implement gradient accumulation. Simulate batch size 256 on GPU with batch size 32 and 8 accumulation steps. Verify loss matches large batch

Day 41 — Ordinary Least Squares

- **LeetGPU (CUDA):** Ordinary Least Squares
- **PyTorch:** Solve OLS using `torch.linalg.lstsq`
- **Project Work:** Implement learning rate warmup + cosine decay scheduler. Add W&B logging for distributed runs (only log from rank 0)

Day 42 — Logistic Regression

- **LeetGPU (CUDA):** Logistic Regression
- **PyTorch:** Implement logistic regression training from scratch

- **Project Work:** Add checkpointing — save model, optimizer, scheduler, epoch, random states. Implement resume from checkpoint

Day 43 — Monte Carlo Integration

- **LeetGPU (CUDA):** Monte Carlo Integration
- **PyTorch:** Implement MC sampling with GPU tensors
- **Project Work:** Implement FSDP (Fully Sharded Data Parallel). Wrap model with FSDP. Compare memory usage: DDP vs FSDP. Benchmark throughput

Day 44 — Matrix Power

- **LeetGPU (CUDA):** Matrix Power
- **PyTorch:** Implement matrix exponentiation
- **Project Work:** Build scaling analysis. Plot: throughput vs number of GPUs (1, 2, 4 if available). Calculate scaling efficiency. Identify communication bottleneck

Day 45 — Nearest Neighbor

- **LeetGPU (CUDA):** Nearest Neighbor
- **PyTorch:** Implement KNN with distance matrix
- **Project Work:** Write a custom C++ extension for PyTorch using `torch.utils.cpp_extension`. Implement a fused operation (e.g., fused LayerNorm+Dropout, or fused bias+GELU). Benchmark against pure PyTorch version

Day 46 — 2D Jacobi Stencil

- **LeetGPU (CUDA):** 2D Jacobi Stencil
- **PyTorch:** Implement as 2D convolution with fixed kernel
- **Project Work:** Write CUDA extension for PyTorch. Implement a custom CUDA kernel (e.g., fused attention or custom reduction). Load via `torch.utils.cpp_extension.load_inline`

Day 47 — Merge Sorted Arrays + Parallel Merge

- **LeetGPU (CUDA):** Merge Sorted Arrays, Parallel Merge
- **PyTorch:** Understand sorting operations on GPU
- **Project Work:** Comprehensive benchmarking. Create table: Single GPU vs DDP (2 GPU) vs DDP+AMP vs FSDP — throughput, memory, time per epoch, scaling efficiency

Day 48 — Stream Compaction

- **LeetGPU (CUDA):** Stream Compaction
- **PyTorch:** Implement `torch.masked_select` equivalent
- **Project Work:** Write comprehensive README. Include architecture diagram, benchmark charts, scaling analysis, C++/CUDA extension documentation

Day 49 — Sparse MatMul + Linear Recurrence

- **LeetGPU (CUDA):** Sparse Matrix-Dense Matrix Multiplication, Linear Recurrence
- **PyTorch:** Implement sparse operations
- **Project Work:** Final cleanup. Ensure all benchmarks are reproducible. Push to GitHub

Day 50 — 2D FFT + Fast Fourier Transform

- **LeetGPU (CUDA):** 2D FFT, Fast Fourier Transform
- **PyTorch:** Implement using `torch.fft`
- **Project Work:** Polish all 4 repos. Cross-link them. Update your resume with all 4 projects

Week 8-10 Checkpoint: Project 4 complete. You've built a distributed training framework with DDP, FSDP, AMP, gradient accumulation, custom C++/CUDA extensions. 55 LeetGPU problems done.

REMAINING LEETGPU PROBLEMS (Days 51-60 — Mop Up)

These didn't fit neatly into projects but complete your LeetGPU profile:

| Day | Problems |

|-----|-----|

| 51 | Rainbow Table, Password Cracking (cryptography — just for LeetGPU completeness) |

| 52 | Simple Inference (easy win) |

| 53 | Count 2D Array Element, Count 3D Array Element |

| 54 | 2D Subarray Sum, 3D Subarray Sum |

| 55 | Max Subarray Sum |

| 56 | Sorting (Hard) |

| 57 | Radix Sort (Hard) |

| 58 | K-Means Clustering (Hard) |

| 59 | BFS Shortest Path, All-Pairs Shortest Paths (Hard) |

| 60 | Multi-Agent Simulation (Hard) |

FINAL DELIVERABLES

4 GitHub Repos:

1. ****nlp-transformer-finetuning**** — DeBERTa multi-label classification, ablations, deployed API
2. ****vision-training-profiling**** — ResNet vs ViT, GPU profiling comparison

3. ****llm-lora-finetuning**** — QLoRA on Mistral/Llama, evaluation harness, vLLM deployment
4. ****distributed-training-framework**** — DDP, FSDP, AMP, C++/CUDA extensions, scaling analysis

LeetGPU Profile:

- 65+ problems solved in both PyTorch and CUDA
- Public solutions repo on GitHub

Updated Resume:

- 4 new ML training projects replacing gaps
- LeetGPU profile link showing CUDA mastery
- Combined narrative: "I train models, I understand them at the GPU level, I optimize and deploy them"

RULES

1. Type every line of project code yourself. Ask AI to explain concepts, not write code
2. Every LeetGPU problem: solve in CUDA first, then PyTorch. Understand the mapping
3. Log every training run to W&B. No training without experiment tracking
4. Every repo gets a README with: problem, approach, results table, how to reproduce
5. Push code daily. Green squares on GitHub matter
6. If you're stuck for more than 2 hours on something, ask for help. Don't spin
7. Skip nothing. The plan is sequential — each day builds on the last